

Autonomous Cognitive Engine for Deep Research and Long- Horizon Tasks

Presented by,
Divyadharshini.S

Overview:

- AI-based system for complex, long-running research workflows
- Designed to maintain context, planning, and execution over time
- Uses a multi-agent, state-driven architecture
- Automates research-oriented tasks such as:
 - Information retrieval
 - Summarization
 - Task tracking
 - Memory management

Problem Statement:

Why This Project?

- Traditional LLM systems:
 - Lose context in long conversations
 - Lack structured planning
 - Produce inconsistent outputs
- Manual research workflows are:
 - Time-consuming
 - Difficult to track
 - Error-prone

Approach & System Architecture:

- Agent-Driven Modular Approach
 - System follows a Supervisor–Worker architecture
 - Each agent has a clear, dedicated responsibility
 - Execution is state-driven and sequential
 - Shared state ensures context continuity
- Core Components
 - Supervisor Agent
 - Worker Agent
 - Virtual File System (VFS)
 - LangGraph-based workflow orchestration

Agents :

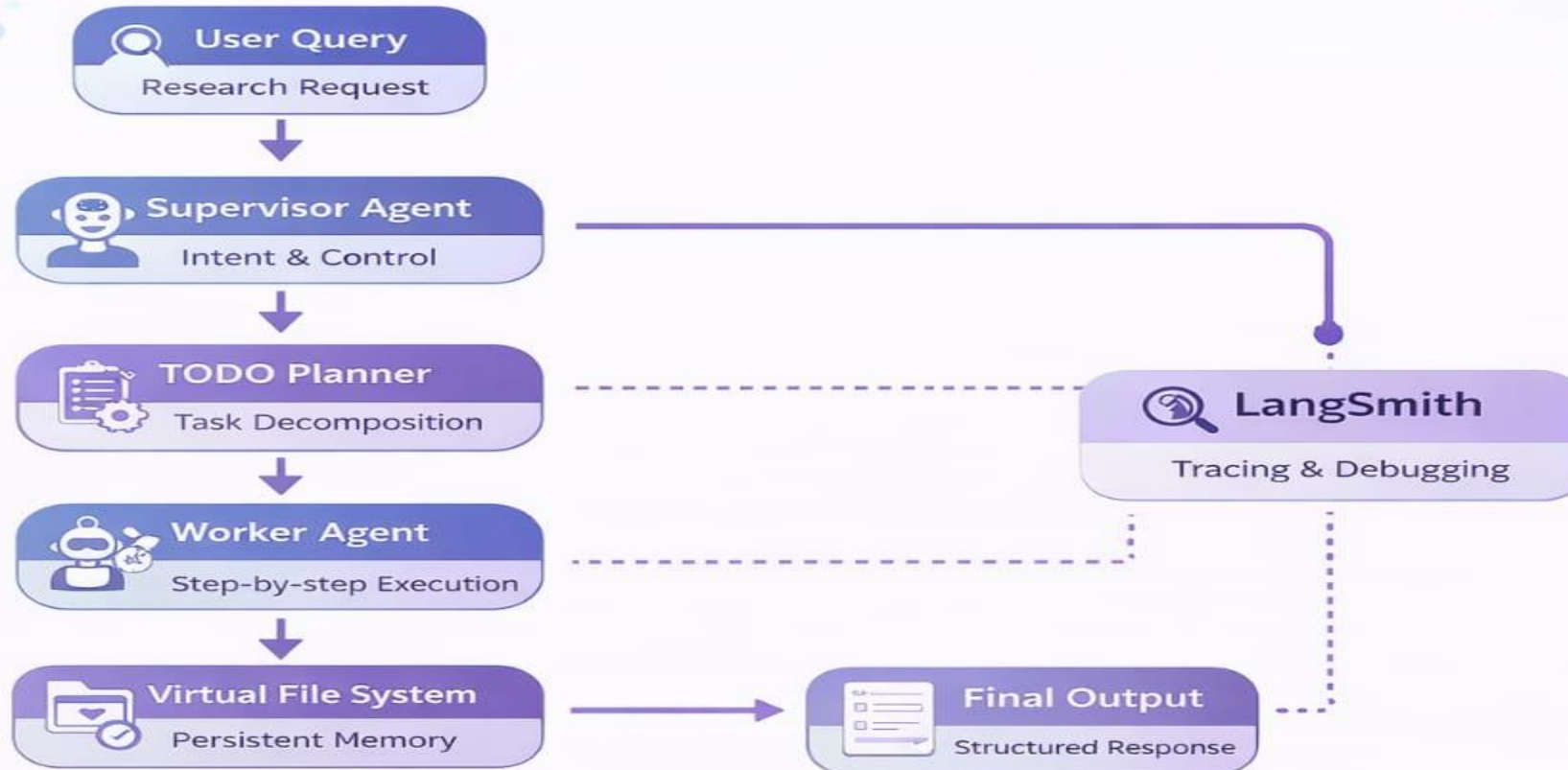
Agent Responsibilities

- **Supervisor Agent**
 - Interprets user intent
 - Generates structured TODO plans
 - Controls execution flow
- **Worker Agent**
 - Executes tasks step-by-step
 - Stores intermediate outputs
 - Updates shared state

Workflow Execution:

Execution Flow of Autonomous Cognitive Agent

A state-driven, explainable workflow for long-horizon tasks



Technology Stack:

Tools & Technologies Used

- Programming Language:
- Python 3.10+
- Agent Frameworks: LangChain, LangGraph
- LLM Integration: Groq API (LLaMA 3.1)
- Environment Management: python-dotenv
- Observability: LangSmith
- User Interface: Streamlit
- Version Control: Git & GitHub (branch-based workflow)

User Interface (Streamlit UI):

Interactive User Experience

- Chat-based interface
- Real-time agent responses
- Persistent session state
- Clear visualization of task execution
- UI Advantage
 - Easy interaction with autonomous agents
 - Suitable for both research and automation use cases

Streamlit User Interface:

The screenshot displays the Streamlit user interface for the 'AI Multi-Agent Chatbot'. The browser window shows the URL 'localhost:8502'. The interface is divided into a left sidebar and a main content area.

Left Sidebar:

- Current Task:** plan a birthday party for my mom and provide some websites for it and also summarize it all
- Delegation Trace:**
 - > Planner → Planner Tool
 - > Search → Search Tool
 - > Summarizer → Summarizer Tool

Main Content Area:

- AI Multi-Agent Chatbot** (with a robot icon)
- A message from the user: "plan a birthday party for my mom and provide some websites for it and also summarize it all"
- A response from the 'Planner' tool: **Birthday Party Plan for Your Mom**

To plan a memorable birthday party for your mom, consider the following steps:

 - Theme and Guest List:** Decide on a theme that your mom will love, such as a garden party, a vintage tea party, or a spa day. Create a guest list of close family and friends who will appreciate the celebration.
 - Date and Time:** Choose a date and time that works best for your mom's schedule. Make sure to give her enough notice so she can prepare and feel special on her special day.
 - Venue:** Decide on a venue that fits the theme and guest list. It could be a private room at a restaurant, a backyard, or a local park.
 - Food and Drinks:** Plan a menu that your mom will enjoy. You can opt for a buffet, a sit-down dinner, or even a dessert-only party. Don't forget to consider any dietary restrictions.
 - Decorations and Entertainment:** Add a personal touch with decorations and entertainment that fit the theme. You can hire a DJ, a photographer, or even a magician to make the party more memorable.
- An input field at the bottom with the placeholder text "Ask something..." and a send button (upward arrow).

The Windows taskbar at the bottom shows the time as 20:34 on 20-01-2026, with system icons for language (ENG IN), network, and battery.

LangSmith Observability & Debugging

LangSmith Integration

- Tracks agent execution paths
- Visualizes:
 - Task decomposition
 - LLM calls
 - State transitions
- Helps in:
 - Debugging recursion issues
 - Improving prompt design
 - Understanding agent behavior
- Benefit
 - Makes the system transparent and explainable

Langsmith Tracing:

The screenshot displays the LangSmith Tracing interface for a project named 'milestone3subagent'. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for 'LangSmith', 'Application' (All applications, Home, Tracing, Monitoring, Datasets & Experiments, Annotation Queues, Prompts, Playground, Studio, Deployments), 'Documentation', 'Invitations', 'Theme', 'Settings', and 'Personal' (sdivyadharshinisaravan...).
- Tracing List:** A table showing a list of runs for the 'milestone3subagent' project. The table has columns for 'Name' and 'Input'. The first run is highlighted.
- Trace View:** A detailed view of the selected run, showing a waterfall chart of the execution flow. The flow includes: 'Main Agent Run' (3.64s, 1.3K), 'Supervisor Agent' (3.56s, 1,291), 'Planner Agent' (0.00s), 'Planner Tool' (0.00s), 'Search Agent' (1.69s, 609), 'Search Tool' (1.69s, 609), 'Chat...' (1.69s, 609), 'Summarizer Agent' (1.26s, 682), 'Summarizer Tool' (1.26s, 682), and 'Chat...' (1.26s, 682).
- Main Agent Run Detail:** A detailed view of the 'Main Agent Run' showing the 'Input' (User Input: plan a birthday party for my mom and provide some websites for it and also summarize it all), 'State' (Current Task: plan a birthday party..., Final Answer: **Party Plan for You..., Messages: [...]), 'Todo Status' (Agent: Planner, Output: Skipped, Status: Skipped, Task: plan a birthday party..., Tool: Planner Tool), and 'Metadata' (START TIME: 01/20/2026, 06:53:36 PM GMT+5; END TIME: 01/20/2026, 06:53:40 PM GMT+5; STATUS: Success; TOTAL TOKENS: 1,291 tokens; LATENCY: 3.64s; TYPE: Chain).

The bottom of the image shows a Windows taskbar with various application icons and a system clock indicating 19:02 on 20-01-2026.

Challenges Faced:

- Managing long-term context within LLM limits
- State recursion errors in LangGraph workflows
- Coordinating shared state across multiple agents

Future Scope:

- Parallel execution using multiple worker agents
- Distributed cloud-based memory
- Integration with external tools (web search, APIs)
- Advanced task prioritization and scheduling

Conclusion:

- Demonstrates a scalable autonomous agent system
- Effectively combines:
 - Planning
 - Reasoning
 - Execution
 - Persistent memory
- Provides a strong foundation for advanced AI research and automation



Q & A



THANK YOU!